

Cómo se construyó — Cuadrante

Actualizado el 25 de septiembre de 2026

Resumen

Cuadrante nació como un artifact dentro de Claude: cada sincronización con Canvas disparaba una sesión de Claude, y eso consumía la suscripción cada 4-6 horas aunque nadie estuviera mirando. La segunda versión resolvió eso: una app independiente que no usa IA para nada de lo que se puede hacer con código normal. Esta versión añade dos piezas nuevas que sí usan IA, pero de forma puntual y barata (capa gratuita de Gemini, un puñado de llamadas al día o al mes, nunca por sesión de usuario): una Agenda que te dice qué hacer cada día, y un catálogo de retos de hábitos saludables que crece solo.

La arquitectura en una frase: una página web estática (HTML/JS) + Firebase (quién entra y dónde se guardan los datos) + varios robots en GitHub Actions (Canvas y Blackboard cada 4h, la Agenda cada mañana, los retos una vez al mes) + Gemini como IA gratuita puntual para esas dos piezas nuevas. Todo gratis, todo puede ampliarse a más personas sin tocar la base de datos.

Qué ha cambiado desde la primera versión (23 de septiembre)

Multi-usuario. Los datos pasaron de vivir sueltos en Firestore a un espacio por persona:

`users/{email}/...` . Esto permitió añadir una segunda persona (Blackboard/UAH) sin que una vea los datos de la otra, con `sync_blackboard.py` como "hermano" de `sync_canvas.py` .

`migrate_to_users.py` fue el script de un solo uso que copió los datos viejos al nuevo formato; ya se puede borrar una vez comprobado que todo carga bien.

Pestaña Agenda. Un agente diario (IA) que reparte las tareas pendientes en "qué hacer hoy", y que se puede complementar con apuntes propios. Ver la sección dedicada más abajo.

Catálogo de retos. Una colección compartida en Firestore de frases de hábitos saludables, sembrada una vez y ampliada sola cada mes con IA. Ver la sección dedicada más abajo — **todavía no tiene interfaz en la app**, es una pieza de datos lista para cuando se decida mostrarla.

Avisos push. Ya no es una idea futura: `sync_canvas.py` , `sync_blackboard.py` y `generate_agenda.py` mandan notificaciones push reales (entregas de hoy, eventos nuevos de Blackboard, cosas de la Agenda que vencen mañana), usando `pywebpush` y una clave VAPID.

Fallo corregido: tras pasar a multi-usuario, `index.html` se quedó leyendo y escribiendo en las rutas planas antiguas (`tasks` , `avisos` ...) en vez de `users/{email}/tasks` , etc. Como `firestore.rules` ya solo

permitía las rutas nuevas, las pestañas Tareas, Avisos y Horario no podían leer ni guardar nada hasta que se corrigió.

Stack tecnológico

CAPA	TECNOLOGÍA	PARA QUÉ
Frontend	HTML + CSS + JavaScript (sin framework)	La app en sí, un único <code>index.html</code>
Autenticación	Firebase Authentication (correo/contraseña)	Que solo las cuentas autorizadas puedan entrar
Base de datos	Firebase Firestore (capa gratuita Spark)	Guardar tareas, avisos, horario, agenda y el catálogo de retos
Sincronización	Python 3.12 (<code>requests</code> , <code>firebase-admin</code> , <code>icalendar</code> , <code>pywebpush</code>)	Leer Canvas/Blackboard y escribir en Firestore, mandar avisos push
IA (opcional)	Google Gemini API, modelo <code>gemini-3.5-flash-lite</code> (capa gratuita)	Escribir el plan diario de la Agenda y generar retos nuevos
Programación sola	GitHub Actions (cron)	Ejecutar cada robot en su horario, sin intervención
Publicación	GitHub Pages	Servir la web con una URL fija, instalable como PWA

Estructura de archivos

ARCHIVO	QUÉ HACE
<code>index.html</code>	La app completa: interfaz, login y lectura/escritura en Firestore en tiempo real
<code>sync_canvas.py</code>	Robot de Canvas: llama a su API y actualiza <code>users/{email}/...</code>
<code>sync_blackboard.py</code>	Robot "hermano" para una segunda persona con Blackboard (mismo Firestore, otro espacio de datos)
<code>generate_agenda.py</code>	El agente diario: para cada persona, calcula qué tareas están activas y le pide a Gemini una frase de "qué hacer hoy" por cada una
<code>seed_retos.py</code>	Siembra el catálogo de retos una vez (de un solo uso, seguro de repetir)
<code>generate_retos.py</code>	Cada mes le pide a Gemini 20 retos nuevos, evitando duplicar los que ya hay
<code>migrate_to_users.py</code>	Migración de un solo uso al formato multi-usuario — ya se puede borrar
<code>firestore.rules</code>	Reglas de seguridad: cada persona solo lee/escribe su propio <code>users/{su-email}/</code> ; el catálogo de retos lo lee cualquiera con sesión

	iniciada
<code>.github/workflows/sync.yml</code>	Sincroniza Canvas cada 4h
<code>.github/workflows/sync-blackboard.yml</code>	Sincroniza Blackboard cada hora
<code>.github/workflows/generate-agenda.yml</code>	Genera la Agenda cada mañana (6:00 UTC)
<code>.github/workflows/seed-retos.yml</code>	Siembra el catálogo (a mano, una vez)
<code>.github/workflows/generate-retos.yml</code>	Amplía el catálogo el día 1 de cada mes

Sincronización con Canvas y Blackboard

`sync_canvas.py` hace dos llamadas a la API oficial de Canvas (no al feed ICS, que es menos fiable para esto):

1. `/api/v1/planner/items` — todas las entregas próximas; se filtra por `plannable_type` (`assignment` , `quiz` , `discussion_topic`) y por los `course_id` de las asignaturas reales.
2. `/api/v1/announcements` — los anuncios de esas mismas asignaturas.

Cada `course_id` se traduce a un nombre de asignatura legible mediante el diccionario `COURSE_MAP` . Antes de escribir nada, el script lee lo que ya existe en Firestore para no pisar el `done` de una tarea o el `leído` de un aviso marcado a mano — solo actualiza título, fecha y enlace. Las escrituras van en lotes (`ChunkedBatch`) para no superar el límite de 500 operaciones por lote de Firestore.

`sync_blackboard.py` sigue el mismo patrón pero lee el feed `.ics` de "suscribirse" del calendario de Blackboard (la URL funciona como contraseña, nunca se imprime ni se guarda en ningún sitio salvo el secreto de GitHub) y escribe en el espacio de la otra persona (`users/{su-email}/...`), sin que ninguna vea los datos de la otra.

Modelo de datos

RUTA	DOCUMENTO (ID)	CAMPOS PRINCIPALES
<code>users/{email}/tasks</code>	<code>t-<tipo>-<id></code> / <code>bb-<hash></code>	title, course, due_at, url, type, source, done
<code>users/{email}/avisos</code>	<code>a-<id></code>	title, message, course, url, posted_at, leído
<code>users/{email}/schedule</code>	autogenerado	subject, day, room, start, end, color (lo edita la persona, el robot no lo toca)
<code>users/{email}/agenda</code>	<code>auto-<id-tarea></code> (agente) / autogenerado (personal)	texto, día, done, tipo (tarea/personal), task_id, vencimiento, fuente, creado, hecho_el

<code>users/{email}/settings</code>	<code>colors</code>	color elegido por asignatura
<code>users/{email}/pushSubscriptions</code>	autogenerado	endpoint, keys (suscripción push del navegador)
<code>users/{email}/meta</code>	<code>sync</code>	marcas de última sincronización y de avisos ya mandados hoy
<code>retos</code>	<code>r-<hash del texto></code>	texto, categoría (si es de la siembra), fuente (seed/gemini), creado — catálogo compartido, sin dueño

Cómo funciona la Agenda (el agente diario)

`generate_agenda.py` corre una vez al día (6:00 UTC) y recorre la colección `users/` entera, así que sirve a la vez para cualquier persona que use la app, sin duplicar código por persona.

Para cada una: mira sus tareas con `done=false` y fecha de entrega dentro de los próximos 21 días (constante `HORIZONTE_DIAS`), o ya vencidas sin límite hacia atrás. Con esa lista (solo título, asignatura y días restantes — nunca el enunciado, que no guarda), le pide a Gemini una frase corta por tarea, en formato JSON estricto (`response_schema` de la API, para no depender de que la IA "se porte bien" con el formato). Guarda cada frase en `users/{email}/agenda/auto-{id-tarea}`: si el documento ya existe, solo refresca el texto y la fecha de vencimiento; si es la primera vez, lo crea con `done:false`.

La app (`index.html`) no vuelve a preguntar por días futuros: calcula en el propio navegador qué mostrar "hoy" con una regla simple — todo lo no marcado como hecho sigue visible mientras no haya pasado su fecha de vencimiento, sin que el robot tenga que "mover" nada de un día a otro. Eso es lo que hace que algo pendiente se quede ahí varios días seguidos hasta que se complete o venza.

El mismo script manda un aviso push si algo de la Agenda (de una tarea o de un apunte propio con fecha) vence al día siguiente y sigue sin marcarse hecho, con protección para no mandar dos avisos el mismo día si se relanza a mano.

Catálogo de retos de hábitos saludables

`seed_retos.py` siembra, de un solo uso, unos 96 retos cortos (hábitos saludables con emoji, en español) en la colección `retos` — compartida entre todas las personas que usen la app, sin dueño individual.

`generate_retos.py` corre el día 1 de cada mes: lee el catálogo actual, le pide a Gemini 20 retos nuevos "distintos a estos que ya existen" (con la lista de los ya existentes en el propio prompt), y los añade evitando duplicados tanto exactos como parafraseados (comparando texto normalizado — sin emoji, acentos ni mayúsculas — antes de decidir si algo ya existía).

Este catálogo hoy es solo datos: no hay ninguna pantalla en `index.html` que lo muestre todavía. Es la base lista para, cuando se decida, añadir una pestaña o un aviso push con "el reto del día" sin tener que rehacer nada de esto.

Seguridad

Nunca se guarda la contraseña de Canvas ni la URL de Blackboard en ningún sitio legible. Canvas usa un token de acceso personal (revocable desde Canvas sin cambiar la contraseña); Blackboard usa la URL secreta de "suscribirse" al calendario, que funciona como una contraseña. Ambos viven únicamente como secretos cifrados de GitHub Actions (`CANVAS_TOKEN` , `BLACKBOARD_ICS_URL`) — no están en el código ni en Firestore.

`firestore.rules` obliga a que quien lea o escriba dentro de `users/{email}/` esté identificado exactamente con ese email en su sesión; cualquier otra cuenta se rechaza. El catálogo compartido `retos` se puede leer con cualquier sesión iniciada, pero nadie puede escribir en él desde el cliente — solo el robot, con la clave de administrador de Firebase, que se salta las reglas.

El inicio de sesión usa el proveedor de correo electrónico/contraseña de Firebase Authentication (no "Iniciar sesión con Google"): cada persona tiene su propia cuenta y contraseña creadas directamente en Firebase, y el propio código (`ALLOWED_EMAIL`) rechaza cualquier otra cuenta que no sea la suya.

La clave de Gemini (`GEMINI_API_KEY`) vive igual que las demás, como secreto de GitHub Actions: no da acceso a Firestore ni a ningún dato personal, solo permite generar texto dentro de la capa gratuita.

El Cuadrante original en Claude

El primer Cuadrante (el artifact dentro de Claude) no se ha borrado. Ahora solo aloja dos secciones que siguen dependiendo de IA a propósito, porque generar una newsletter o buscar cursos requiere investigar en la web:

- Newsletter de los lunes (IA, informática, política, mundo, datos curiosos).
- Sugerencias de cursos gratuitos con certificado para LinkedIn.

La pestaña Más de la app nueva enlaza directamente ahí. Las tareas programadas que generaban estas dos secciones automáticamente se desactivaron (consumían la suscripción); si en algún momento se quiere que vuelvan a generarse solas, se puede reactivar bajo demanda.

Cómo ampliarlo

- **Asignatura o cuatrimestre nuevo:** edita el diccionario `COURSE_MAP` al principio de `sync_canvas.py` con los `course_id` nuevos (se sacan de `/api/v1/courses` con el token) y sube el archivo de nuevo a GitHub.
- **Una persona más:** crea su cuenta en Firebase Authentication, un script de sincronización a su medida (tipo `sync_blackboard.py`) con su propio `OWNER_EMAIL` , y una copia de `index.html` con `ALLOWED_EMAIL` apuntando a ella — las reglas de Firestore ya están listas para cualquier número de personas sin tocarlas.
- **Cambiar la frecuencia de un robot:** edita el `cron` del workflow correspondiente en `.github/workflows/` .
- **Mostrar el catálogo de retos:** ya existen los datos en Firestore (`retos`); falta solo una pantalla o un aviso push que elija uno al azar (o el que menos se haya mostrado) y lo enseñe.
- **Otras ideas:** un widget de "próxima tarea" en la pantalla de inicio del móvil, resúmenes semanales por push, o volver a activar Newsletter/Cursos de forma automática llamando a una IA gratuita en vez de Claude.